



Chapter 16

Project 5.2: Simple Multivariate Statistics

Are variables related? If so what's the relationship? An analyst tries to answer these two questions. A negative answer — the null hypothesis — doesn't require too many supporting details. A positive answer, on the other hand, suggests that a model can be defined to describe the relationship. In this chapter, we'll look at simple correlation and linear regression as two elements of modeling a relationship between variables.

In this chapter, we'll expand on some skills of data analysis:

- Use of the built-in `statistics` library to compute correlation measures and linear regression coefficients.
- Use of the `matplotlib` library to create images. This means creating plot images outside a Jupyter Lab environment.
- Expanding on the base modeling application to add features.

This chapter's project will expand on earlier projects. Look back at [***Chapter 13, Project 4.1: Visual Analysis Techniques***](#) for some of the graphical techniques used in a Jupyter Lab context. These need to be more fully automated. The project will add multivariate statistics and graphs to illustrate relationships among variables.

16.1 Description

In ***Chapter 15, Project 5.1: Modeling Base Application*** we created an application to create a summary document with some core statistics. In that application, we looked at univariate statistics to characterize the data distributions. These statistics included measurements of the location, spread, and shape of a distribution. Functions like mean, median, mode, variance, and standard deviation were emphasized as ways to understand location and spread. The characterization of shape via skewness and kurtosis was left as an extra exercise for you.

The base application from the previous chapter needs to be expanded to include the multivariate statistics and diagrams that are essential for clarifying the relationships among variables. There are a vast number of possible functions to describe the relationships among two variables. See

<https://www.itl.nist.gov/div898/handbook/pmd/section8/pmd8.htm> for some insight into the number of choices available.

We'll limit ourselves to linear functions. In the simplest cases, there are two steps to creating a linear model: identifying a correlation and creating the coefficients for a line that fits the data. We'll look at each of these steps in the next two sections.

16.1.1 Correlation coefficient

The coefficient of correlation measures how well the values of two variables correlate with each other. A value of 1 indicates perfect correlation; a value of zero indicates no discernable correlation. A value of -1 indicates an “anti-correlation”: when one variable is at its maximum value, the other variable is at its minimum.

See ***Figure 16.1*** to see how the correlation coefficient describes the distribution of the two variables.

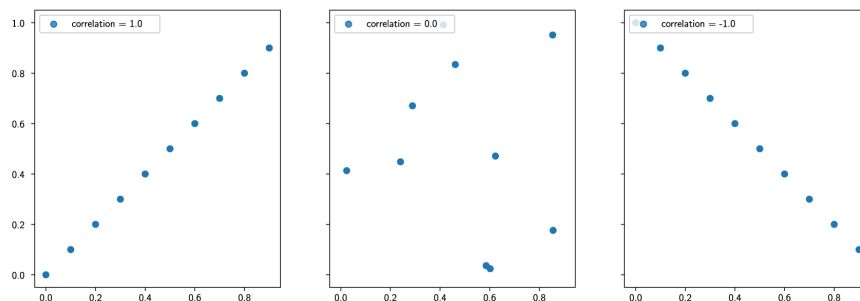


Figure 16.1: Correlation Coefficients

The computation of the coefficient compares individual values of variables, X_i and Y_i , to the mean values for those variables, $\bar{X}$ and $\bar{Y}$. Here's a formula:

$$r = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2} \sqrt{\sum (Y_i - \bar{Y})^2}}$$

The computations of the mean values, $\bar{X}$ and $\bar{Y}$, can be factored into this, creating a somewhat more complicated version that's often used to create the coefficient in a single pass through the data.

This function is available as `statistics.correlation()` in the `statistics` library.

If two variables correlate with each other, then a linear function will map one variable to a value near the other variable. If the correlation is 1.0 or -1.0, the mapping will be exact. For other correlation values, the mapping will be close, but not exact. In the next section, we'll show how to transform the correlation coefficient into the parameters for a line.

16.1.2 Linear regression

One equation for a line is $y = mx + b$. The values of m and b are parameters that describe the specific linear relationship between the x and y variables.

When fitting a line to data, we're estimating the parameters for a line. The goal is to minimize the error between the line and the actual data. The “least squares” technique is often used.

The two coefficients, b and m , can be computed as follows:

$$b = \frac{\sum X_i^2 \sum Y_i^2 - \sum X_i \sum X_i Y_i}{n \sum X_i^2 - (\sum X_i)^2}$$

$$m = \frac{n \sum X_i Y_i - \sum X_i \sum Y_i}{n \sum X_i^2 - (\sum X_i)^2}$$

This is available as `statistics.linear_regression()` in the standard library. This saves us from having to write these two functions.

The various sums and sums of squares are not terribly difficult values to compute. The built-in `sum()` function is the basis for most of this. We can use `sum(map(lambda x: x^2, x_values))` to compute $\sum X_i^2$.

To clarify these multivariate relationships, diagrams can be very helpful. In the next sections, we'll look at the most important type of diagram that needs to be part of the overall application.

16.1.3 Diagrams

One essential diagram for showing multivariate data is the X-Y “scatter” plot. In [*Chapter 13, Project 4.1: Visual Analysis Techniques*](#) we looked at ways to create these. In that chapter, we relied on Jupyter Lab to present the diagram as part of the overall web page. For this application, we'll need to embed the diagram into a document.

This generally means there will be a markup document that includes a reference to a diagram file. The format of the diagram file can be SVG, PNG, or even JPEG. For technical graphics, the SVG files are often the smallest and scale extremely well.

Each markup language, including Markdown, RST, HTML, and LaTeX, have unique ways to identify the place where an image needs to be inserted. In the case of Markdown, it's often necessary to use HTML syntax to properly include frames and captions.

Now that we've seen what the application needs to do, we can look at an approach to create the software.

16.2 Approach

As with the previous project, this application works in these two distinct parts:

1. Compute the statistics and create the diagram files.
2. Create a report file in a simplified markup language from a template with the details interpolated. A tool like **Jinja** is very helpful for this.

Once the report file in a markup language — like Markdown or RST — is available, then a tool like **Pandoc** can be used to create an HTML page or a PDF document from the markup file. Using a tool like **Pandoc** permits quite a bit of flexibility in choosing the final format. It also allows the insertion of style sheets and page templates in a tidy, uniform way.

The LaTeX language as markup provides the most comprehensive capabilities. It is challenging to work with, however. Languages like Markdown and RST are designed to offer fewer, easier-to-use capabilities.

This book is written with LaTeX.

We'll look at three aspects of this application: the statistical computations, creating the diagrams, and finally, creating the final markup file

to include the diagrams. We'll start with a quick review of the statistical computations.

16.2.1 Statistical computations

The statistical summary output file, in TOML notation, has a section for each variable and the univariate statistics about those variables.

This section of the file looked like the following snippet of TOML:

```
[x.location]
  mean = 9.0
[x.spread]
  variance = 11.0
[y.location]
  mean = 7.5
[y.spread]
  variance = 4.125
```

When parsed, the TOML syntax of `x.location` and `x.spread` creates a dictionary that looks like the following fragment of a Python object:

```
{
    some metadata here...

    'x': {
        'location': {
            'mean': 9.0
        },
        'spread': {
            'variance': 11.0
        }
    },
    'y': {
        etc.
    }
}
```

This structure can be expanded to include additional locations and spread statistical measures. It can also be expanded to include multivariate statistics. The `statistics` module has `correlation()` and `covariance()` functions, making it easy to include these measures.

For datasets with few variables, it's common to consider a matrix that includes all the combinations of covariance between variables. This leads to two alternative representations of these additional statistics:

- A separate section for a covariance matrix. A section label of `[covariance]` can be followed by nested dictionaries that include all combinations of variables. Since the covariance matrix is symmetric, all n^2 combinations aren't needed; only $n \times (n - 1)$ values are unique.
- Multivariate sub-sections within each variable's section. This means we'd have `x.location`, `x.spread`, `x.covariance.y`, and `x.correlation.y` sub-sections for the `x` variable.

For a dataset with fewer variables, it seems sensible to bundle covariance and correlation into the details for a given variable. In the case of Anscombe's Quartet, with only two variables, the covariance and correlation seem like they belong with the other statistics.

For a dataset with a larger number of variables, the covariance among all the variables can become bewildering. In these cases, a technique like finding principal components might be needed to reduce the number of variables to a more manageable population. In this case, separate sections with covariance and auto-correlation might be more useful.

The resulting model is often the result of some careful thought, based on the covariance matrix. For this reason, a separate `[model]` section should be provided with some details about the model's structure and the coefficients. In the case of a linear model, there are two coefficients, sometimes called β_0 and β_1 . We've called them b and m .

For Python 3.11, the included `tomllib` module doesn't create TOML-format files. It's, therefore, necessary to properly format a text file that can be parsed by the `tomllib` module. It's helpful to use a Jinja template for this.

16.2.2 Analysis diagrams

Diagrams must first be created. Once created, they can then be included in a document. The process of creating a diagram is nearly identical to the approach used in Jupyter Lab. A few extra steps need to be taken to export the diagram to a file that can be imported into a document.

When working in Jupyter Lab, some cells to load the data are required to create two variables, `x` and `y`, with the values to be plotted. After these cells, a cell like the following example will create and display a scatter plot:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

# Labels and Title
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('Series I')

# Draw Scatter
_ = ax.scatter(x, y)
```

This presumes previous cells have loaded clean data and extracted two list objects, `x` and `y`, with the values to be plotted.

The above code sample doesn't save the resulting PNG or SVG file, however. To save the figure, we need to perform two more steps. Here are the lines of code required to create a file from the plot:

```
plt.savefig('scatter_x_y.png')  
plt.close(fig)
```

It helps to transform this cell's code into a function. This function has proper type annotations so that a tool like **mypy** can confirm the types are used properly. It can also have unit test cases to be sure it really works.

The `savefig()` function will write a new file in PNG format with the image. If the file path suffix is `'.jpg'` then an SVG format file will be created.

The size of the figure is defined by the `figure()` function. There are often design and page layout considerations that suggest an appropriate size for a figure. This decision can be deferred, and the size can be provided by the markup used to create a final PDF file or HTML page. It's often best, however, to create the figure in the required size and resolution to avoid any unexpected alterations as part of the final publication.

Once the diagram has been created, the Markdown needs to refer to the diagram's PNG or SVG file so it can be included in a document. We'll look at some examples of this in the next section.

16.2.3 Including diagrams in the final document

Diagrams are included in the final document by using markup commands to show where the diagram should be placed, and providing other information about captions and sizing.

The Markdown language has a tidy format for the simplest case of including an image in a document:

```
![Alt text to include!](path/to/file.png "Figure caption")
```

Depending on the style sheet, this may be perfectly acceptable. In some cases, the image is the wrong size for its role in the document. Markdown permits using HTML directly instead of the `![image!](path)` construct. Including a diagram often looks like this:

```
<figure>
  
  <figcaption>Figure caption</figcaption>
</figure>
```

Using HTML permits more control over image size and placement via references to CSS.

When using RST, the syntax offers more options without switching to HTML. Including a diagram would be like this:

```
.. figure:: path/to/file.png
   :height: 8cm
   :alt: Alt text to include

   Figure caption
```

Using this kind of markup technique creates considerable freedom. The report's author can include content from a variety of sources. This can include boilerplate text that doesn't change, the results of computations, some text based on the computations, and important diagrams.

The formatting of the markup has little impact on the final document. The way a browser renders HTML depends on the markup and the style sheets, not the formatting of the source file. Similarly, when creating a PDF document, this is often done by LaTeX tools, which create the final document based on LaTeX settings in the document's preamble.

Now that we have an approach, we can look at the deliverable files that must be built.

16.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for model module classes in the `tests` folder.
- Mock objects for the `csv_extract` module tests that will be part of the unit tests.
- Unit tests for the `csv_extract` module components that are in the `tests` folder.
- An application to extend the summary written to a TOML file, including figures with diagrams.
- An application secondary feature to transform the TOML file to an HTML page or PDF file with the summary.

We'll look at a few of these deliverables in a little more detail. We'll start with some suggestions for creating the acceptance tests.

16.3.1 Acceptance tests

As we noted in the previous chapter's section on acceptance testing, **Acceptance testing**, the output TOML document can be parsed and examined by the `Then` steps of a scenario. Because we're looking at Anscombe's Quartet data in the examples in this book, a subset of data for testing doesn't really make much sense. For any other dataset, a subset should be extracted and used for acceptance testing.

It is often helpful to extract a small subset that's used for acceptance testing. Instead of processing millions of rows, a few dozen rows are adequate to confirm the application read and summarized data. The

data should be representative of the entire set of samples under consideration.

This subset is part of the testing suite; as such, it rarely changes. This makes the results predictable.

The secondary feature of this application — expanding on the TOML output to add extensive Markdown — also works with text files. This makes it relatively easy to create scenarios to confirm the correct behavior by reading and writing text files. In many cases, the `Then` steps will look for a few key features of the resulting document. They may check for specific section titles or a few important keywords included in boilerplate text. Of course, the test scenario can check for substitution values that are computed and are part of the TOML summary.

The automated testing can't easily confirm that the document makes sense to prospective readers. It can't be sure the colors chosen for the figures make the relationships clear. For this kind of usability test, a good copy editor or trusted associate is essential.

16.3.2 Unit tests

A unit test for a function to create a figure can't do very much. It's limited to confirming that a PNG or SVG file was created. It's difficult for an automated test to “look” at the image to be sure it has a title, labels for the axes, and sensible colors.

It is important not to overlook the unit test cases that confirm output files are created. A figure that looks great in a Jupyter notebook will not get written to a file unless the CLI application saves the figure to a file.

For some applications, it makes sense to mock the `plt` package functions to be sure the application calls the right functions with the expected argument values. Note that a mocked version of

`plt.subplots()` may need to return a tuple with several `Mock` objects.

We'll need to define a complex collection of mock objects to form the fixture for testing. The fixture creation can look like the following example:

```
@fixture
def mocked_plt_module(monkeypatch):
    fig_mock = Mock()
    ax_mock = Mock(
        set_xlabel=Mock(),
        set_ylabel=Mock(),
        set_title=Mock(),
        scatter=Mock(),
    )
    plt_mock = Mock(
        subplots=Mock(
            return_value=(fig_mock, ax_mock)
        ),
        savefig=Mock(),
        close=Mock()
    )
    monkeypatch.setattr(summary_app, 'plt', plt_mock)
    return plt_mock, fig_mock, ax_mock
```

This fixture creates three mock objects. The `plt_mock` is a mock of the overall `plt` module; it defines three mock functions that will be used by the application. The `fig_mock` is a mock of the figure object returned by the `subplots()` function. The `ax_mock` is a mock of the axes object, which is also returned by the `subplots()` function. This mocked axes object is used to provide axis labels, and the title, and perform the scatter plot request.

This three-tuple of mock objects is then used by a test as follows:

```
def test_scatter(mocked_plt_module):
    plt_mock, fig_mock, ax_mock = mocked_plt_module
```

```
summary_app.scatter_figure([sentinel.X], [sentinel.Y])

assert plt_mock.subplots.mock_calls == [call()]
assert plt_mock.savefig.mock_calls == [call('scatter_x_y.png')]
assert plt_mock.close.mock_calls == [call(fig_mock)]
```

This test function evaluates the application's `scatter_figure()` function. The test function then confirms that the various functions from the `plt` module are called with the expected argument values.

The test can continue by looking at the calls to the `ax_mock` object to see if the labels and title requests were made as expected. This level of detail — looking at the calls to the axes object — may be a bit too fine-grained. These tests become very brittle as we explore changing titles or colors to help make a point more clearly.

The overall use of mock objects, however, helps make sure the application will create the needed file with an image.

16.4 Summary

In this chapter, we've extended the automated analysis and reporting to include more use of the built-in `statistics` library to compute correlation and linear regression coefficients. We've also made use of the **matplotlib** library to create images that reveal relationships among variables.

The objective of automated reporting is designed to reduce the number of manual steps and avoid places where omissions or errors can lead to unreliable data analysis. Few things are more embarrassing than a presentation that reuses a diagram from the previous period's data. It's far too easy to fail to rebuild one important notebook in a series of analysis products.

The level of automation needs to be treated with a great deal of respect. Once a reporting application is built and deployed, it must be ac-

tively monitored to be sure it's working and producing useful, informative results. The analysis job shifts from developing an understanding to monitoring and maintaining the tools that confirm — or reject — that understanding.

In the next chapter, we'll review the journey from raw data to a polished suite of applications that acquires, cleans, and summarizes the data.

16.5 Extras

Here are some ideas for you to add to this project.

16.5.1 Use **pandas** to compute basic statistics

The **pandas** package offers a robust set of tools for doing data analysis. The core concept is to create a **DataFrame** that contains the relevant samples. The **pandas** package needs to be installed and added to the **requirements.txt** file.

There are methods for transforming a sequence of **SeriesSample** objects into a **DataFrame**. The best approach is often to convert each of the **pydantic** objects into a dictionary, and build the dataframe from the list of dictionaries.

The idea is something like the following:

```
import pandas as pd

df = pd.DataFrame([dict(s) for s in series_data])
```

In this example, the value of **series_data** is a sequence of **SeriesSample** instances.

Each column in the resulting dataframe will be one of the variables of the sample. Given this object, methods of the **DataFrame** object pro-

duce useful statistics.

The `corr()` function, for example, computes the correlation values among all of the columns in the dataframe.

The `cov()` function computes the pairwise covariance among the columns in the dataframe.

Pandas doesn't compute the linear regression parameters, but it can create a wide variety of descriptive statistics.

See <https://pandas.pydata.org> for more information on Pandas.

In addition to a variety of statistics computations, this package is designed for interactive use. It works particularly well with Jupyter Lab. The interested reader may want to revisit [*Chapter 13, Project 4.1: Visual Analysis Techniques*](#) using Pandas instead of native Python.

16.5.2 Use the dask version of pandas

The **pandas** package offers a robust set of tools for doing data analysis. When the volume of data is vast, it helps to process parts of the dataset concurrently. The **Dask** project has an implementation of the **pandas** package that maximizes opportunities for concurrent processing.

The `dask` package needs to be installed and added to the `requirements.txt` file. This will include a `pandas` package that can be used to improve overall application performance.

16.5.3 Use numpy for statistics

The **numpy** package offers a collection of tools for doing high-performance processing on large arrays of data. These basic tools are en-

hanced with libraries for statistics and linear algebra among many, many other features. This package needs to be installed and added to the `requirements.txt` file.

The **numpy** package works with its own internal array type. This means the `SeriesSample` objects aren't used directly. Instead, a `numpy.array` object can be created for each of the variables in the source series.

The conversion might look like the following:

-

```
import numpy as np

x = np.array(s.x for s in series_data)
y = np.array(s.y for s in series_data)
```

In this example, the value of `series_data` is a sequence of `SeriesSample` instances.

It's also sensible to create a single multi-dimensional array. In this case, axis 0 (i.e. rows) will be the individual samples, and axis 1 (i.e. columns) will be the values for each variable of the sample.

An array has methods like `mean()` to return the mean of the values. When using a multi-dimensional array, it's essential to provide the `axis=0` parameter to ensure that the results come from processing the collection of rows:

```
import numpy as np

a = np.array([s.x, s.y] for s in series_data)
print(f"means = {a.mean(axis=0)}")
```

See

<https://numpy.org/doc/stable/reference/routines.statistics.html#>

Using the least squares technique to compute the coefficients for a line can be confusing. The least squares solver in **numpy.linalg** is a very general algorithm, which can be applied to creating a linear model. The `numpy.linalg.lstsq()` function expects a small matrix that contains the "x" values. The result will be a vector with the same length as each of the "x" matrices. The "y" values will also be a vector.

The processing winds up looking something like the following:

```
import numpy as np

A = np.array([[s.x, 1] for s in series_data])
y = np.array([s.y for s in series_data])
m, b = np.linalg.lstsq(A, y, rcond=None)[0]
print(f"y = {m:.1f}x + {b:.1f}")
```

The value of `A` is a small matrix based on the x values. The value of `y` is a simple array of the y values. The least-squares algorithm returns a four-tuple with the coefficients, residuals, the rank of the source matrix, and any singular values. In the above example, we only wanted the vector of the coefficients, so we used `[0]` to extract the coefficient values from the four-tuple with the results.

This is further decomposed to extract the two coefficients for the line that best fits this set of points. See:

<https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html>.

This approach has a distinct advantage when working with very large sets of data. The **numpy** libraries are very fast and designed to scale to extremely large data volumes.

16.5.4 Use **scikit-learn** for modeling

The **scikit-learn** library has a vast number of tools focused on modeling and machine learning. This library is built on the foundation of

numpy, so both packages need to be installed.

The data needs to be converted into **numpy** arrays. Because the modeling approach is very generalized, the assumption is there may be multiple independent variables that predict the value of a dependent variable.

The conversion might look like the following:

```
import numpy as np

x = np.array([s.x for s in series_data])
y = np.array([s.y for s in series_data])
```

In this example, the value of `series_data` is a sequence of `SeriesSample` instances. The `x` array uses a very short vector for each sample; in this case, there's only a single value. It needs to be a vector to fit the generalized least-squares regression that **scikit-learn** is capable of solving.

The scikit-learn library is designed to create models in a very generalized way. The model isn't always a simple line with a coefficient and an intercept that define the relationship. Because of this very general approach to modeling, we'll create an instance of the `linear_model.LinearRegression` class. This object has methods to create coefficients that fit a given set of data points. We can then examine the coefficients, or use them to interpolate new values.

The code might look like the following:

```
from sklearn import linear_model

reg = linear_model.LinearRegression()
reg.fit(x, y)
print(f"y = {reg.coef_[0]:.1f}x + {reg.intercept_:.1f}")
```

The linear model's `coef_` attribute is a vector of coefficients, the same length as each row of the `x` independent variable values. Even when the row length is 1, the result is a vector with a length of 1.

Because this works with **numpy** it can work with very large sets of data. Further, the scikit-learn approach to creating models to fit data generalizes to a number of machine-learning approaches. This is often the next step in creating richer and more useful models.

16.5.5 Compute the correlation and regression using functional programming

The computations for correlation and the coefficients for a line can be summarized as follows. First, we'll define a function $M(a; f())$ that computes the mean of a transformed sequence of values. The $f()$ function transforms each value, a_i . An identity function, $\phi(a_i) = a_i$, does no transformation:

$$M(a; f()) = \frac{1}{N} \sum f(a_i)$$

We'll also need a function to compute the standard deviation for a variable, a .

$$S(a) = \sqrt{\frac{\sum (a_i - \bar{a})^2}{N}}$$

This lets us define a number of related values as mean values after some transformation.

$$\begin{aligned}\bar{x} &= M(x; f(a_i) = a_i) \\ \bar{y} &= M(y; f(a_i) = a_i) \\ \overline{x^2} &= M(x; f(a_i) = a_i^2) \\ \overline{y^2} &= M(y; f(a_i) = a_i^2) \\ \overline{xy} &= M(x, y; f(a_i, b_i) = a_i \times b_i)\end{aligned}$$

From these individual values, we can compute the correlation coefficient, r_{xy} .

$$r_{xy} = \frac{\overline{xy} - \bar{x}\bar{y}}{\sqrt{(\overline{x^2} - \bar{x}^2)(\overline{y^2} - \bar{y}^2)}}$$

In addition to the above values, we need two more values for the standard deviations of the two variables.

$$s_x = S(x)s_y = S(y)$$

From the correlation coefficient, and the two standard deviations, we can compute the coefficient of the line, m , and the intercept value, b .

$$m = r_{xy} \frac{s_y}{s_x}$$
$$b = \bar{y} - m\bar{x}$$

This yields the coefficient, m , and intercept, b , for the equation $y = mx + b$, which minimizes the error between the given samples and the line. This is computed using one higher-order function, $M(a;f())$, and one ordinary function, $S(a)$. This doesn't seem to be a significant improvement over other methods. Because it's built using standard library functions and functional programming techniques, it can be applied to any Python data structure. This can save the step of transforming data into **numpy** array objects.